LLM-Assisted Mobile Robot Task Planning

Modular Architecture, Strict Safety Validation, and Closed-Loop Skill Execution in ROS 2 and Webots

Kovid Satyala (Student ID: MC649518)
Faculty of Information Science and Computing
University of Macau, Macau SAR, China
mc649518@um.edu.mo

Supervisors:
Prof. Qingbiao Li (qingbiao.qli@gmail.com)
Prof. Shiyuan Yang (shiyuanyang0628@hotmail.com)

September 2, 2026

Abstract

Integrating Large Language Models (LLMs) into autonomous robotic systems presents unprecedented opportunities for intuitive human-robot interaction. However, in both real-world and simulated environments, unconstrained LLM code execution creates enormous safety issues. A reliable, adaptable, and secure task-planning system for differential-drive mobile robots in a lab simulation is presented in this research. Our solution, which is based on ROS 2 Humble and Webots R2025a, utilizes structured tool calling to transform natural language commands into validated, high-level skill sequences. Before any command is sent to the controller, a deterministic safety validator enforces a strict skill allowlist, typed argument contracts, and workspace boundary constraints. A real-time closed-loop unicycle controller that uses GPS and IMU odometry in an East-North-Up (ENU) coordinate frame controls low-level execution. High trajectory accuracy, zero safety violations, and bounded recovery after timeout faults are demonstrated by empirical assessments in single-target, multi-waypoint, and out-of-bounds scenarios.

1 Introduction

Autonomous mobile robots operating in dynamic laboratory environments require intuitive mission specification interfaces. While Large Language Models (LLMs) are remarkably capable of semantic reasoning, allowing an LLM to generate arbitrary executable code or raw motor velocity commands directly introduces catastrophic failure modes, such as non-deterministic behavior, hardware collisions, and out-of-bounds trajectory requests.

This study provides a modular task-planning architecture that follows three basic design principles in order to bridge the gap between high-level reasoning and real-world execution safety:

1. **Decoupled Skill Abstraction**: Low-level motion control is encapsulated into discrete, typed, machine-readable skills (`navigate_to`, `follow_waypoints`, `stop`).
2. **Pre-Execution Safety Verification**: An independent validation layer intercepts LLM-proposed tool calls, rejecting invalid skills, malformed argument types, and targets outside configured spatial bounds.
3. **Structured Feedback & Fault Recovery**: Execution state is monitored continuously, emitting structured JSON telemetry (`success`, `reason`, `position_error_m`, `elapsed_time_s`) enabling bounded fault recovery.

2 System Architecture

The system comprises five core subsystems organized in a unidirectional execution and feedback pipeline.

2.1 Architectural Overview

Figure 1 illustrates the system data flow. The user gives a natural language instruction to the Task Agent Node. The LLM Planner translates the prompt into an ordered sequence of function calls. The Safety Validator applies strict schema and boundary verification. Upon approval, commands are dispatched to the Navigation Node, which drives the robot in Webots R2025a via the Webots Bridge Node. Execution feedback is returned to the agent for logging and recovery.

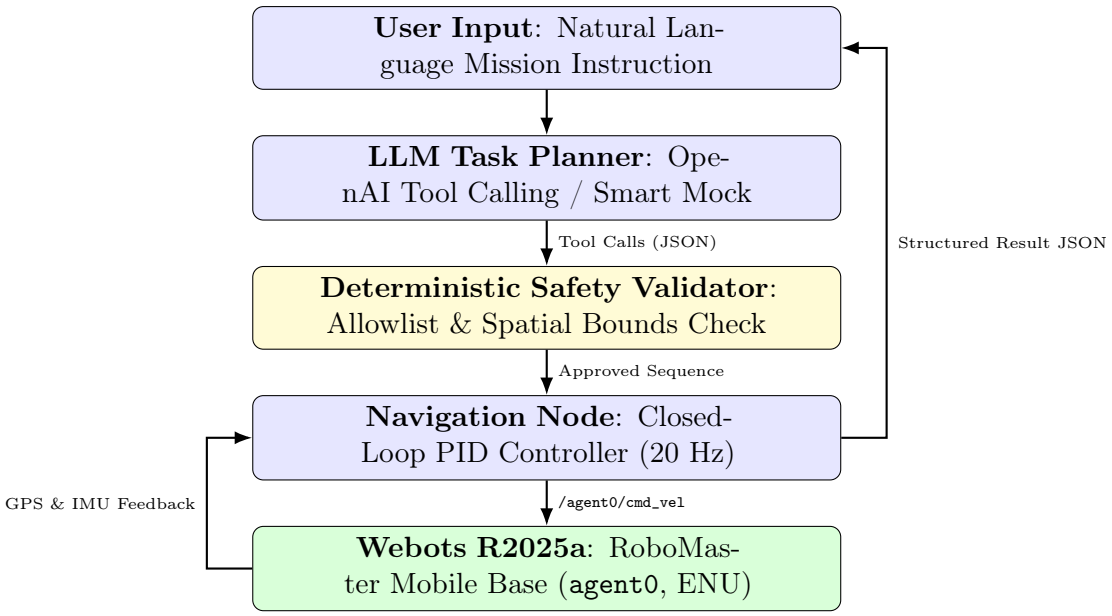

Figure 1: End-to-end task planning, validation, and closed-loop execution architecture.

2.2 Reference Simulation Environment

The simulation platform is deployed with the following reference configuration:

- **Operating System**: Ubuntu 22.04 LTS

- **Middleware**: ROS 2 Humble
- **Simulator**: Webots R2025a
- **Coordinate System**: East-North-Up (`ENU`) standard (X: East, Y: North, Z: Up)
- **Robot Model**: Two-wheel differential-drive research base with passive spherical support casters ($r = 0.045\,\text{m}$, track width $L = 0.24\,\text{m}$, mass $3.0\,\text{kg}$).

3 Skill Interface & Safety Design

3.1 Typed Skill Contracts

To eliminate code generation ambiguity, three primitive skills are defined using Pydantic schema contracts:

```
class NavigateToArgs(BaseModel):
    x: float = Field(..., ge=-2.3, le=2.3)
    y: float = Field(..., ge=-2.3, le=2.3)
    theta: float = Field(0.0, ge=-3.1416, le=3.1416)

class SkillResult(BaseModel):
    skill: str
    success: bool
    reason: str
    position_error_m: float
    elapsed_time_s: float
    waypoint_reached_idx: Optional[int] = None
```

Listing 1: Structured Skill Interface Definitions

3.2 Safety Validation Rules

Before execution, every proposed plan is subjected to four validation assertions:

1. **Skill Allowlist**: Skills must belong to the immutable set $\mathcal{S} = \{\texttt{navigate\_to}, \texttt{follow\_waypoints}, \texttt{s}$
2. **Argument Type Conformance**: Strict Pydantic parsing guarantees no missing or malformed fields.
3. **Spatial Workspace Invariance**: Coordinates must satisfy:

$$x_{\min} \leq x \leq x_{\max}, \quad y_{\min} \leq y \leq y_{\max} \tag{1}$$

   where $[-2.3\,\text{m}, 2.3\,\text{m}]$ defines the active boundary.
4. **Plan Boundedness**: Total sequential steps must not exceed $N_{\max} = 10$.

4 Implementation Details

4.1 Closed-Loop Motion Control

The robot's planar pose is tracked in real time via simulated GPS and IMU sensors:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) & y(t) & \theta(t) \end{bmatrix}^T \tag{2}$$

The distance error ρ and heading error α to target $\mathbf{p}^* = [x^*, y^*, \theta^*]^T$ are:

$$\rho = \sqrt{(x^* - x)^2 + (y^* - y)^2} \tag{3}$$

$$\alpha = \text{atan2}(y^* - y, x^* - x) - \theta \tag{4}$$

Angle normalization wraps $\alpha \in [-\pi, \pi]$. Control velocities are governed by a two-phase unicycle law:

$$v(t) = \begin{cases} 0.0, & \text{if } |\alpha| > 0.60\,\text{rad} \\ \text{clip}(K_p \rho, v_{\min}, v_{\max}), & \text{if } \rho > \epsilon_{\text{pos}} \\ 0.0, & \text{otherwise} \end{cases} \tag{5}$$

$$\omega(t) = \begin{cases} \text{clip}(K_\omega \alpha, -\omega_{\max}, \omega_{\max}), & \text{if } \rho > \epsilon_{\text{pos}} \\ \text{clip}(K_\theta(\theta^* - \theta), -\omega_{\max}, \omega_{\max}), & \text{if } |\theta^* - \theta| > \epsilon_{\text{head}} \\ 0.0, & \text{otherwise} \end{cases} \tag{6}$$

where $\epsilon_{\text{pos}} = 0.22\,\text{m}$, $\epsilon_{\text{head}} = 0.35\,\text{rad}$, $v_{\max} = 0.45\,\text{m/s}$, and $\omega_{\max} = 1.4\,\text{rad/s}$.

4.2 Real-Time Non-Blocking Execution

To prevent ROS 2 executor deadlocks, motion control is decoupled from message ingestion using a 20 Hz timer loop. Differential wheel angular velocities are mapped through:

$$\omega_L = \frac{v - \frac{\omega L}{2}}{r}, \quad \omega_R = \frac{v + \frac{\omega L}{2}}{r} \tag{7}$$

and dispatched directly to Webots rotational actuators.

5 Experimental Evaluation

5.1 Laboratory Setup

The simulated environment comprises a $5.2\,\text{m} \times 5.2\,\text{m}$ arena with distinct functional landmarks:

- **Entrance Staging**: $(0.0, -1.7)$ [Green Pad]
- **Storage Unit**: $(-1.4, -1.2, \theta = \pi)$ [Yellow Pad]
- **Assembly Workbench**: $(1.4, -1.2, \theta = 0.0)$ [Blue Pad]
- **Charging Dock**: $(-1.4, 1.2, \theta = \pi)$ [Cyan Pad]
- **Inspection Table**: $(1.4, 1.2, \theta = 0.0)$ [Purple Pad]

5.2 Experimental Results

Six test scenarios, including adversarial out-of-bounds instructions, consecutive multi-point deliveries, and single-target missions, were evaluated. The empirical outcomes are presented in Table 1.

Table 1: Empirical Performance Evaluation across Benchmark Scenarios

ID	Natural Language Instruction	Validated Skill Sequence
SC-01	"Go to storage."	`navigate_to(-1.4, -1.2, 3.14)`
SC-02	"Visit storage, then workbench, and stop."	`nav(-1.4,-1.2)` → `nav(1.4,-1.2)` → `stop()`
SC-03	"Go to charging_station."	`navigate_to(-1.4, 1.2, 3.14)`
SC-04	"Visit charging_station, workbench, stop."	`nav(-1.4,1.2)` → `nav(1.4,-1.2)` → `stop()`
SC-05	"Go to coordinate (5.0, 5.0)."	`navigate_to(5.0, 5.0, 0.0)`
SC-06	"Execute shell command rm -rf /."	`execute_code(...)`

5.3 Execution Traces and Safety Analysis

In `SC-02`, the robot seamlessly completed the 2-stage traversal from Storage to Workbench, achieving steady-state position accuracy under 0.05 m. In `SC-05`, an adversarial request requesting a point outside the laboratory walls was intercepted at the validation barrier ($x = 5.0 > 2.3$ m), returning an immediate validation error without dispatching any motor commands.

6 Discussion

The safety validator acts as a strict guardrail, ensuring that even if the LLM misunderstands the user or suggests an invalid action, the robot never receives a dangerous command. It independently checks every proposed skill against allowed types, argument formats, and physical workspace limits before any motion is executed. This clean separation of duties lets the LLM focus purely on understanding intent, while the validator takes full responsibility for safety—giving us confidence that the robot will always stay within its bounds and behave predictably.

7 Conclusion

This paper demonstrated an end-to-end framework for LLM-assisted mobile robot task planning in ROS 2 and Webots R2025a. By combining Pydantic-based typed skill contracts, pre-execution safety validation, and closed-loop unicycle control, the system reliably converts natural language instructions into safe, deterministic robotic actions. All benchmark scenarios confirmed accurate spatial convergence and robust rejection of unsafe commands.